

Customer Support Ticket Analyzer

1. Introduction

Customer support teams handle a large number of service tickets daily, and analyzing these tickets helps identify recurring issues, customer sentiment, and areas for improvement. This project builds a Python-based **Ticket Analysis System** that can store, clean, analyze, and extract insights from customer support tickets. The goal is to demonstrate skills in **data handling, text processing, keyword analysis, and summary statistics** using Python.

2. Implementation

a. Print the initial ticket data in a readable format:

```
1 # Initial ticket data
2
3 ticket_data = {
4     'Ticket_No': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
5     'Customer_Name': ['Ravi', 'Meera', 'Sam', 'Anu', 'Rakesh', 'Divya', 'Arjun', 'Kiran', 'Leela', 'Nisha'],
6     'Issue_Description': [
7         'Internet not working!!! ',
8         'slow response, very poor service ',
9         'GREAT support! issue resolved.',
10        'okay... need help ',
11        'not BAD but slow', 'Excellent guidance, Very Helpful!',
12        'good support and good behaviour!',
13        'Poor handling of technical issue',
14        'Satisfied. Could be better.',
15        'Good service... quick response.'
16    ],
17     'Priority': ['High', 'Low', 'High', 'Medium', 'Low', 'High', 'Medium', 'High', 'Low', 'Medium']
18 }
19
20 import pandas as pd
21 df = pd.DataFrame(ticket_data)
22 print(df)
```

	Ticket_No	Customer_Name	Issue_Description	Priority
0	1	Ravi	Internet not working!!!	High
1	2	Meera	slow response, very poor service	Low
2	3	Sam	GREAT support! issue resolved.	High
3	4	Anu	okay... need help	Medium
4	5	Rakesh	not BAD but slow	Low
5	6	Divya	Excellent guidance, Very Helpful!	High
6	7	Arjun	good support and good behaviour!	Medium
7	8	Kiran	Poor handling of technical issue	High
8	9	Leela	Satisfied. Could be better.	Low
9	10	Nisha	Good service... quick response.	Medium

b. Add More Tickets:

```
1 # Add new tickets
2
3 def add_tickets(ticket_data, n):
4     for i in range(n):
5         name = input("Enter Customer Name: ")
6         issue = input("Enter Issue Description: ")
7         priority = input("Enter Priority (High/Medium/Low): ")
8
9         while priority not in ['High', 'Medium', 'Low']:
10             priority = input("Invalid! Enter Priority (High/Medium/Low): ")
11
12         ticket_data['Ticket_No'].append(len(ticket_data['Ticket_No'])+1)
13         ticket_data['Customer_Name'].append(name)
14         ticket_data['Issue_Description'].append(issue)
15         ticket_data['Priority'].append(priority)
16
17 add_tickets(ticket_data, 2)
18 print(pd.DataFrame(ticket_data))
```

Enter Customer Name: Arun
Enter Issue Description: GREAT support! issue resolved.
Enter Priority (High/Medium/Low): High
Enter Customer Name: David
Enter Issue Description: good support and good behaviour!
Enter Priority (High/Medium/Low): Medium

	Ticket_No	Customer_Name	Issue_Description	Priority
0	1	Ravi	Internet not working!!!	High
1	2	Meera	slow response, very poor service	Low
2	3	Sam	GREAT support! issue resolved.	High
3	4	Anu	okay... need help	Medium
4	5	Rakesh	not BAD but slow	Low
5	6	Divya	Excellent guidance, Very Helpful!	High
6	7	Arjun	good support and good behaviour!	Medium
7	8	Kiran	Poor handling of technical issue	High
8	9	Leela	Satisfied. Could be better.	Low
9	10	Nisha	Good service... quick response.	Medium
10	11	Arun	GREAT support! issue resolved.	High
11	12	David	good support and good behaviour!	Medium

c. Text Cleaning for Issue Descriptions:

```
1 # Text Cleaning for Issue Descriptions
2
3 import re
4 def clean_text(text):
5     text = text.lower()
6     text = re.sub(r'^\w\s', '', text)
7     text = re.sub(r'\s+', ' ', text)
8     text = text.strip()
9     text = text.replace("ok", "okay")
10    return text
11
12 ticket_data['Issue_Description'] = [clean_text(desc) for desc in ticket_data['Issue_Description']]
13 print(pd.DataFrame(ticket_data))
```

	Ticket_No	Customer_Name	Issue_Description	Priority
0	1	Ravi	internet not working	High
1	2	Meera	slow response very poor service	Low
2	3	Sam	great support issue resolved	High
3	4	Anu	okayay need help	Medium
4	5	Rakesh	not bad but slow	Low
5	6	Divya	excellent guidance very helpful	High
6	7	Arjun	good support and good behaviour	Medium
7	8	Kiran	poor handling of technical issue	High
8	9	Leela	satisfied could be better	Low
9	10	Nisha	good service quick response	Medium
10	11	Arun	great support issue resolved	High
11	12	David	good support and good behaviour	Medium

d. Keyword-Based Issue Insights:

```
1 # Keyword-Based Insights
2
3 def count_tickets_with_word(word):
4     word = word.lower()
5     return sum(1 for desc in ticket_data['Issue_Description'] if word in desc)
6
7 print("Tickets with 'poor':", count_tickets_with_word("poor"))
8 print("Tickets with 'good':", count_tickets_with_word("good"))
9 print("Tickets with 'slow':", count_tickets_with_word("slow"))
10 print("Tickets with 'excellent':", count_tickets_with_word("excellent"))
11
```

```
... Tickets with 'poor': 2
Tickets with 'good': 3
Tickets with 'slow': 2
Tickets with 'excellent': 1
```

e. Final Summary & Insights:

i) Priority Analysis

```
1 # Final Summary & Insights
2
3 # Priority Analysis
4 from collections import Counter
5 priority_count = Counter(ticket_data['Priority'])
6 print(priority_count)
7
```

```
Counter({'High': 5, 'Medium': 4, 'Low': 3})
```

ii) Ticket With the Longest Issue Description

```
1 # Ticket With the Longest Issue Description
2
3 lengths = [len(desc.split()) for desc in ticket_data['Issue_Description']]
4 max_index = lengths.index(max(lengths))
5
6 print("Ticket No:", ticket_data['Ticket_No'][max_index])
7 print("Customer:", ticket_data['Customer_Name'][max_index])
8 print("Issue:", ticket_data['Issue_Description'][max_index])
9 print("Word Count:", lengths[max_index])
10
```

```
... Ticket No: 2
Customer: Meera
Issue: slow response very poor service
Word Count: 5
```

iii) Extract Unique Words Used

```
1 # Extract Unique Words Used
2 all_words = set(" ".join(ticket_data['Issue_Description']).split())
3 print("Unique word count:", len(all_words))
4 print("Sorted words:", sorted(all_words))
5
```

```
... Unique word count: 31
Sorted words: ['and', 'bad', 'be', 'behaviour', 'better', 'but', 'could', 'excellent', 'good', 'great', 'guidance', 'handling', 'help', 'helpful', 'internet', 'issue', 'need', 'not', 'of', 'okayay', 'poor', 'quick', 'resolved', 'response', 'satisfied', 'service', 'slow', '']
```

iv) Final Cleaned ticket_data

```
1 # Final Cleaned ticket_data
2 df = pd.DataFrame(ticket_data)
3 print("Final Cleaned Ticket Data:\n")
4 print(df.to_string(index=False))
5
```

... Final Cleaned Ticket Data:

Ticket_No	Customer_Name	Issue_Description	Priority
1	Ravi	internet not working	High
2	Meera	slow response very poor service	Low
3	Sam	great support issue resolved	High
4	Anu	okayay need help	Medium
5	Rakesh	not bad but slow	Low
6	Divya	excellent guidance very helpful	High
7	Arjun	good support and good behaviour	Medium
8	Kiran	poor handling of technical issue	High
9	Leela	satisfied could be better	Low
10	Nisha	good service quick response	Medium
11	Arun	great support issue resolved	High
12	David	good support and good behaviour	Medium

3. Conclusion

The Customer Support Ticket Analyzer successfully demonstrated how Python can be used for:

- Data loading and organization (lists, dictionaries).
- Text cleaning and transformation (string operations, regex).
- Keyword-based analysis (functions, loops, conditionals).
- Summary statistics and insights (priority distribution, longest issue, unique words).

This project highlights the importance of structured ticket analysis in improving customer support operations. By automating text cleaning, keyword detection, and statistical summaries, support teams can quickly identify problem areas and enhance service quality.